Institute of Technological Studies of Bizerte

❦

RAIA1 (M1-S1) — Dept. of EE | **AY: 2026-2027**

Midterm Exam | FL and ANN | **Oct 2026**

Teacher: A. Mhamdi | **Time Limit: 60 min**

This exam consists of **9** pages. Please check that it is complete. The **3** tasks are independent and may be solved in any order. The following rules apply:

① **One handwritten, double-sided A4 sheet** is allowed.
② **No electronic material** may be used, except for a basic calculator.
③ **Unjustified or unsupported answers** will not receive full credit.
④ **Round all numerical results** to three decimal places *(i.e., to the nearest thousandth).*
⑤ **Correct answers** receive the points indicated. There is no negative marking.

SOLUTIONS

Task №1 — FOL & CNF

⏳15 min | (4 points)

(a) (2 pts) For each sentence, give the corresponding first-order logic formula. Assume the domain is people with predicates:

- Sx: "x is a student",
- Tx: "x is a teacher",
- Cx: "x is a course",
- $E(x, y)$: "x enrolled in y",
- $L(x, y)$: "x likes y",
- $P(x, y)$: "x passed y".

1. Every student is enrolled in at least one course.
2. There is a teacher who is liked by every student.
3. No student has passed every course.
4. For any student enrolled in a course, there is a teacher whom that student likes.

1. $\forall x(Sx \rightarrow \exists y(Cy \wedge E(x, y)))$
2. $\exists x(Tx \wedge \forall y(Sy \rightarrow L(y, x)))$
3. $\neg\exists x(Sx \wedge \forall y(Cy \rightarrow P(x, y)))$
4. $\forall x\forall y((Sx \wedge Cy \wedge E(x, y)) \rightarrow \exists z(Tz \wedge L(x, z)))$

(b) (2 pts) Convert each first-order formula into **CNF**.

1. $\forall x(Px \rightarrow \exists yQ(x, y))$
2. $\exists x\forall y(P(x, y) \rightarrow \neg\exists zR(x, y, z))$
3. $\forall x(\exists yL(x, y) \leftrightarrow \forall zM(x, z))$
4. $\neg\exists x\forall y(Fx \rightarrow (Gy \wedge H(x, y)))$

1. $\neg P(x) \vee Q(x, f(x))$
   (Skolem function $f(x)$ replaces y, since $\exists y$ is inside the scope of $\forall x$.)

2. $\neg P(a, y) \vee \neg R(a, y, z)$
   ($\exists x$ has no preceding universal, so it becomes the Skolem constant a.)
3. $(\neg L(x, y) \vee M(x, z)) \wedge (\neg M(x, g(x)) \vee L(x, h(x)))$
   ($g(x)$, $h(x)$ are Skolem functions for the two existentials, both depending on x.)
4. $F(x) \wedge (\neg G(f(x)) \vee \neg H(x, f(x)))$
   ($f(x)$ Skolemizes $\exists y$, which depends on the preceding $\forall x$.)

TASK №2 — THE MISSING PROTOTYPE AT SIROCCO LABS

⏳ 25 min | (6 points)

You are a junior security analyst at **Sirocco Labs**, a Tunisian startup based in Bizerte. The company develops an AI-powered translation device for Arabic dialects, and its only working prototype, the **"Jasmin"**, has just disappeared from the secure R&D lab.

This morning, the alarm system flagged the incident. The CEO, **Raed B.**, hands you the incident file and says:

> ! *"Test the hypothesis that Mehdi is the thief."*

You have a rule base *(how things work at **Sirocco Labs**)* and a set of known facts *(what the investigation has established so far)*.

SOLUTIONS

The Rule Base

Rule	IF…	THEN…
$\mathfrak{R}_1$	`HasMotive(x)` ∧ `HadOpportunity(x)` ∧ `HadMeans(x)`	`Suspect(x)`
$\mathfrak{R}_2$	`InDebt(x)`	`HasMotive(x)`
$\mathfrak{R}_3$	`FiredRecently(x)`	`HasMotive(x)`
$\mathfrak{R}_4$	`HasLabAccess(x)` ∧ `WasOnSite(x)`	`HadOpportunity(x)`
$\mathfrak{R}_5$	`HasKeycard(x)`	`HasLabAccess(x)`
$\mathfrak{R}_6$	`KnowsAlarmCode(x)`	`HadMeans(x)`
$\mathfrak{R}_7$	`IsITStaff(x)`	`KnowsAlarmCode(x)`
$\mathfrak{R}_8$	`BadgedIn(x, night)`	`WasOnSite(x)`

Characters

Mehdi IT staff member at Sirocco Labs;
Yasmine former IT staff member, recently let go.

The Known Facts

$\mathfrak{F}_1$ `InDebt(Mehdi)`
$\mathfrak{F}_2$ `HasKeycard(Mehdi)`
$\mathfrak{F}_3$ `BadgedIn(Mehdi, night)`
$\mathfrak{F}_4$ `IsITStaff(Mehdi)`

$\mathfrak{F}_5$ `FiredRecently(Yasmine)`
$\mathfrak{F}_6$ `HasKeycard(Yasmine)`
$\mathfrak{F}_7$ `IsITStaff(Yasmine)`

✎ *There is no fact stating that Yasmine badged in at night.*

Using backward chaining, the rule base and the known facts above, complete the following tasks:

1. Identify which rule has `Suspect(x)` as its conclusion.
2. List the sub-goals that this rule creates.
3. For each sub-goal, find a rule that concludes it, and break it down further.
4. Continue until each leaf is a known fact ($\mathfrak{F}_1$ *to* $\mathfrak{F}_7$).
5. Draw the full **proof tree** *(goal at the top, facts at the bottom).*

(a) (3 pts) Prove the Goal for Mehdi, i.e., `Suspect(Mehdi)`

1. Rule concluding `Suspect(x)`: only $\mathfrak{R}_1$ has `Suspect(x)` as its conclusion.

2. Sub-goals created by $\mathfrak{R}_1$: `HasMotive(Mehdi)`, `HadOpportunity(Mehdi)` and `HadMeans(Mehdi)`. Since $\mathfrak{R}_1$ uses AND, all three must be proven.

3. and 4. Decomposition (goal stack, processed depth-first, left to right):

Step	Goal popped	$\mathfrak{R}/\mathfrak{F}$	Sub-goals pushed
1	`Suspect(Mehdi)`	$\mathfrak{R}_1$	`HasMotive`, `HadOpportunity`, `HadMeans`
2	`HasMotive(Mehdi)`	$\mathfrak{R}_2$	`InDebt(Mehdi)`
3	`InDebt(Mehdi)`	$\mathfrak{F}_1$	none (success)
4	`HadOpportunity(Mehdi)`	$\mathfrak{R}_4$	`HasLabAccess`, `WasOnSite`
5	`HasLabAccess(Mehdi)`	$\mathfrak{R}_5$	`HasKeycard(Mehdi)`
6	`HasKeycard(Mehdi)`	$\mathfrak{F}_2$	none (success)
7	`WasOnSite(Mehdi)`	$\mathfrak{R}_8$	`BadgedIn(Mehdi, night)`
8	`BadgedIn(Mehdi, night)`	$\mathfrak{F}_3$	none (success)
9	`HadMeans(Mehdi)`	$\mathfrak{R}_6$	`KnowsAlarmCode(Mehdi)`
10	`KnowsAlarmCode(Mehdi)`	$\mathfrak{R}_7$	`IsITStaff(Mehdi)`
11	`IsITStaff(Mehdi)`	$\mathfrak{F}_4$	none (success)

For `HasMotive`, two rules exist ($\mathfrak{R}_2$ and $\mathfrak{R}_3$). $\mathfrak{R}_2$ is tried first and succeeds through $\mathfrak{F}_1$, so $\mathfrak{R}_3$ is never needed and no backtracking occurs.

5. Proof tree:

```
1   Suspect(Mehdi)                          [R1]
2   +-- HasMotive(Mehdi)                    [R2]
```

```
3   |   +-- InDebt(Mehdi)                    FACT F1
4   +-- HadOpportunity(Mehdi)                [R4]
5   |   +-- HasLabAccess(Mehdi)              [R5]
6   |   |   +-- HasKeycard(Mehdi)            FACT F2
7   |   +-- WasOnSite(Mehdi)                 [R8]
8   |       +-- BadgedIn(Mehdi, night)       FACT F3
9   +-- HadMeans(Mehdi)                      [R6]
10      +-- KnowsAlarmCode(Mehdi)            [R7]
11          +-- IsITStaff(Mehdi)             FACT F4
```

Conclusion: every leaf of the tree is a known fact, so `Suspect(Mehdi)` is proven. **Mehdi is a suspect.**

(b) (3 pts) Test Yasmine, i.e., `Suspect(Yasmine)`. In case it fails, what additional fact would be needed to make Yasmine a suspect?

1. Rule and sub-goals: $\mathfrak{R}_1$ is the only rule concluding `Suspect(x)`. It creates three sub-goals for Yasmine: `HasMotive(Yasmine)`, `HadOpportunity(Yasmine)` and `HadMeans(Yasmine)`. All three must be proven.

2. Decomposition (goal stack, depth-first, left to right):

Step	Goal popped	$\mathfrak{R}/\mathfrak{F}$	Result
1	`Suspect(Yasmine)`	$\mathfrak{R}_1$	pushes `HasMotive`, `HadOpportunity`, `HadMeans`
2	`HasMotive(Yasmine)`	$\mathfrak{R}_2$	pushes `InDebt(Yasmine)`
3	`InDebt(Yasmine)`	none	FAILS: no fact and no rule concludes it
4	`HasMotive(Yasmine)`	$\mathfrak{R}_3$	backtrack: pushes `FiredRecently(Yasmine)`
5	`FiredRecently(Yasmine)`	$\mathfrak{F}_5$	success
6	`HadOpportunity(Yasmine)`	$\mathfrak{R}_4$	pushes `HasLabAccess`, `WasOnSite`
7	`HasLabAccess(Yasmine)`	$\mathfrak{R}_5$	pushes `HasKeycard(Yasmine)`
8	`HasKeycard(Yasmine)`	$\mathfrak{F}_6$	success
9	`WasOnSite(Yasmine)`	$\mathfrak{R}_8$	pushes `BadgedIn(Yasmine, night)`
10	`BadgedIn(Yasmine, night)`	none	FAILS: no such fact
11	`HadMeans(Yasmine)`	$\mathfrak{R}_6$	pushes `KnowsAlarmCode(Yasmine)`
12	`KnowsAlarmCode(Yasmine)`	$\mathfrak{R}_7$	pushes `IsITStaff(Yasmine)`
13	`IsITStaff(Yasmine)`	$\mathfrak{F}_7$	success

3. Choosing between rules: for `HasMotive`, $\mathfrak{R}_2$ is tried first. It fails because Yasmine is not known to be in debt. We backtrack and try $\mathfrak{R}_3$, which succeeds through $\mathfrak{F}_5$.

4. The failing sub-goal: `WasOnSite(Yasmine)`. Its only rule, $\mathfrak{R}_8$, requires `BadgedIn(Yasmine, night)`, and no fact states that she badged in at night. Having a keycard ($\mathfrak{F}_6$) gives her lab access, but not presence on site. No other rule concludes `WasOnSite`, so there is nothing left to backtrack to.

Consequence: `WasOnSite(Yasmine)` fails, so $\mathfrak{R}_4$ fails, so `HadOpportunity(Yasmine)` fails. Since R1 needs all three conditions (AND), `Suspect(Yasmine)` cannot be proven.

5. Proof tree (FAIL marks a failed branch):

```
1   Suspect(Yasmine)                          [R1]  FAIL
2   +-- HasMotive(Yasmine)                    [R3]  OK (backtrack)
3   |   +-- FiredRecently(Yasmine)            FACT F5
4   +-- HadOpportunity(Yasmine)               [R4]  FAIL
5   |   +-- HasLabAccess(Yasmine)             [R5]  OK
6   |   |   +-- HasKeycard(Yasmine)           FACT F6
7   |   +-- WasOnSite(Yasmine)                [R8]  FAIL
8   |       +-- BadgedIn(Yasmine, night)      no fact: FAIL
9   +-- HadMeans(Yasmine)                     [R6]  OK
10      +-- KnowsAlarmCode(Yasmine)           [R7]
11          +-- IsITStaff(Yasmine)            FACT F7
```

6. Missing fact: `BadgedIn(Yasmine, night)`. With it, $\mathfrak{R}_8$ gives `WasOnSite(Yasmine)`, $\mathfrak{R}_4$ gives `HadOpportunity(Yasmine)`, and $\mathfrak{R}_1$ succeeds.

Conclusion: with the current facts, Yasmine is **not** proven to be a suspect.

RAIA1 (M1-S1) — Dept. of EE	Full Name: ----------
MIDTERM EXAM | FL AND ANN	ID: ----------
Teacher: A. Mhamdi	Class: ----------
Oct 2026	Room: ----------

✂ ----------

TASK №3 ⏳20 min | (10 points)

(a) (0.5 pt) Which component of an expert system contains the domain-specific facts and rules provided by human experts?

- ○ Inference engine
- ○ User interface
- ✓ Knowledge base
- ○ Explanation facility

(b) (0.5 pt) Which component of an expert system justifies to the user how or why a conclusion was reached?

- ○ Knowledge base
- ✓ Explanation facility
- ○ Inference engine
- ○ Working memory

(c) (0.5 pt) Forward chaining is best described as which kind of reasoning?

- ○ Goal-driven
- ○ Query-driven
- ○ Hypothesis-driven
- ✓ Data-driven

(d) (0.5 pt) What is the conjunctive normal form of $(A \wedge B) \rightarrow C$?

✓ $\neg A \vee \neg B \vee C$ ○ $(\neg A \wedge \neg B) \vee C$ ○ $A \vee B \vee C$ ○ $(A \vee B) \wedge \neg C$

(e) (0.5 pt) Given the rules $A \wedge B \rightarrow C$ and $C \rightarrow D$, and the known facts A and B, which facts does forward chaining derive?

✓ Both C and D ○ Only C ○ Only D ○ None

SOLUTIONS

DO NOT WRITE ANYTHING HERE

✂ ----------

(f) (0.5 pt) In an expert system, which inference strategy starts from a goal (hypothesis) and works backwards to find supporting facts?

- ○ Forward chaining
- ✓ Backward chaining
- ○ Knowledge acquisition
- ○ Explanation facility

(g) (0.5 pt) Which first-order logic sentence correctly represents "All humans are mortal"?

- ○ $\forall x(H(x) \land M(x))$
- ○ $\exists x(H(x) \rightarrow M(x))$
- ✓ $\forall x(H(x) \rightarrow M(x))$
- ○ $\exists x(H(x) \land M(x))$

(h) (0.5 pt) Which command activates a local project environment so that `Pkg.add` affects only that project?

- ○ `Pkg.local()`
- ○ `Pkg.env("MyProject")`
- ○ `Pkg.use("MyProject")`
- ✓ `Pkg.activate("path/to/project")`

(i) (0.5 pt) Which Pkg command creates/updates a `Manifest.toml` capturing the exact resolved dependency versions?

○ `Pkg.develop()` ✓ `Pkg.instantiate()` ○ `Pkg.build()` ○ `Pkg.gc()`

(j) (0.5 pt) In the `Pkg` REPL mode *(entered by pressing* `]`*)*, which command adds a new package `MyPkg` to the current environment?

✓ `add MyPkg` ○ `using MyPkg` ○ `install MyPkg` ○ `Pkg.get("MyPkg")`

SOLUTIONS

DO NOT WRITE ANYTHING HERE

✂ ----------

(k) (0.5 pt) What does this code print?

```julia
x = [1, 2, 3, 4]
println(sum(x .^ 2))
```

○ 10 ✓ 30 ○ 100 ○ Error

(l) (0.5 pt) What does this code print?

```julia
function f(n)
    n ⩽ 1 && return 1
    return n * f(n - 1)
end
println(f(5))
```

○ 24 ○ 60 ✓ 120 ○ 720

(m) (0.5 pt) What does this code print?

```julia
d = Dict("a" ⇒ 1, "b" ⇒ 2)
d["c"] = d["a"] + d["b"]
println(length(d))
```

○ 2 ✓ 3 ○ 4 ○ Error

(n) (0.5 pt) What does this code print?

```julia
s = [x for x in 1:10 if x % 3 == 0]
println(s)
```

✓ `[3, 6, 9]` ○ `[0, 3, 6, 9]` ○ `[1, 4, 7, 10]` ○ `[3, 6, 9, 12]`

(o) (0.5 pt) Given

```julia
f(x::Int, y::Int) = "ints"
f(x::Number, y::Number) = "numbers"
```

What does `f(1, 2)` return?

✓ `"ints"` ○ `"numbers"` ○ A `MethodError` ○ An ambiguity error

SOLUTIONS

DO NOT WRITE ANYTHING HERE

✂ ----------

(p) (0.5 pt) What is “multiple dispatch” in Julia?

- ○ Calling multiple functions in a single expression using the ▷ operator
- ○ Running the same function on multiple threads simultaneously
- ✓ Selecting a method based on the runtime types of all arguments
- ○ Overloading operators using macros only

(q) (0.5 pt) What keyword is used to define an immutable composite type in Julia?

✓ `struct` ○ `mutable struct` ○ `type` ○ `immutable`

(r) (0.5 pt) Which keyword combination defines a composite type whose fields **can** be reassigned after construction?

○ `struct` ✓ `mutable struct` ○ `abstract struct` ○ `const struct`

(s) (0.5 pt) What is the purpose of an inner constructor in a Julia struct?

- ○ To define methods that can be dispatched on other types
- ○ To allow the struct to inherit fields from another struct
- ○ To make the struct mutable
- ✓ To enforce invariants or validate/transform fields before construction

(t) (0.5 pt) Which statement about Julia’s type system is correct?

- ○ Concrete types can be subtyped by other concrete types
- ✓ Abstract types cannot be instantiated and exist only to organize the type hierarchy
- ○ Every struct in Julia automatically subtypes Any and nothing else can
- ○ Abstract types can have fields

SOLUTIONS